

# Pre-requisites for Autonomous Systems

Lokesh Mohanty

AiREX Lab

Indian Institute of Science

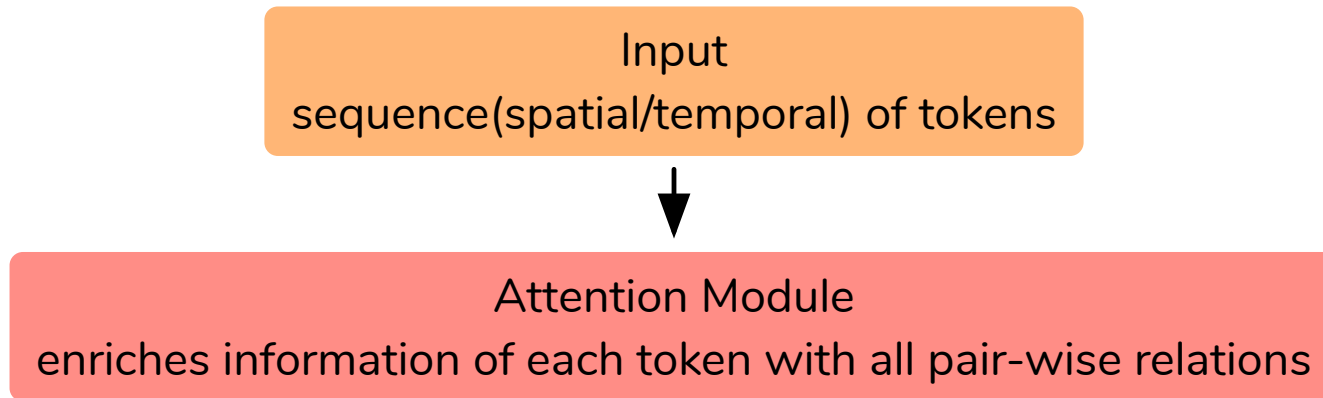
29 Jan, 2026

# Contents

|                                             |   |
|---------------------------------------------|---|
| Pre-requisites for Autonomous Systems ..... | 0 |
| Transformer .....                           | 2 |
| QFormer .....                               | 7 |
| Diffusion .....                             | 8 |
| Neural Operator .....                       | 9 |
| Bibliography .....                          | 9 |

# Transformer

- This architecture<sup>1</sup> was introduced for Seq-to-Seq tasks
- Excels in learning spatial and temporal relations
- Highly parallelizable leading to fast execution



---

<sup>1</sup>[Vaswani et. al.] Attention is all you need [1]

# Transformer

- Three vectors (query( $q_i$ ), key( $k_i$ ), value( $v_i$ )) are generated from each input token( $x_i$ )
- Each token queries ( $q$ ) how much of attention it should give to every other token

# Transformer

- Three vectors (**query**( $q_i$ ), **key**( $k_i$ ), **value**( $v_i$ )) are generated from each input token( $x_i$ )
- Each token **queries** ( $q$ ) how much of **attention** it should give to every other token

$X \rightarrow$  input,  $n \rightarrow$  number of tokens,  $d \rightarrow$  token dimension

$$x_i \rightarrow i^{\text{th}} \text{ token} \in \mathbb{R}^d, W^{\{Q, K, V\}} \in \mathbb{R}^{d \times d}$$

$$q_i \rightarrow x_i W^Q, k_i \rightarrow x_i W^K, v_i \rightarrow x_i W^V$$

$$\text{probs}(p) \leftarrow \text{Softmax} \left( \text{Attention Scores} \leftarrow \frac{q_i K^T}{\sqrt{d}} \right)$$

$$\text{Attention}(a_i) \leftarrow pV$$

# Transformer

- Three vectors (**query**( $q_i$ ), **key**( $k_i$ ), **value**( $v_i$ )) are generated from each input token( $x_i$ )
- Each token **queries** ( $q$ ) how much of **attention** it should give to every other token
- In **Self-Attention**, query and key, value belong to the same sequence
- In **Cross-Attention**, query belongs to a different sequence

$X \rightarrow$  input,  $n \rightarrow$  number of tokens,  $d \rightarrow$  token dimension

$$x_i \rightarrow i^{\text{th}} \text{ token} \in \mathbb{R}^d, W^{\{Q, K, V\}} \in \mathbb{R}^{d \times d}$$

$$q_i \rightarrow x_i W^Q, k_i \rightarrow x_i W^K, v_i \rightarrow x_i W^V$$

$$\text{probs}(p) \leftarrow \text{Softmax} \left( \text{Attention Scores} \leftarrow \frac{q_i K^T}{\sqrt{d}} \right)$$

$$\text{Attention}(a_i) \leftarrow pV$$

# Transformer

- Three vectors (**query**( $q_i$ ), **key**( $k_i$ ), **value**( $v_i$ )) are generated from each input token( $x_i$ )
- Each token **queries** ( $q$ ) how much of **attention** it should give to every other token
- In **Self-Attention**, query and key, value belong to the same sequence
- In **Cross-Attention**, query belongs to a different sequence

$X \rightarrow$  input,  $n \rightarrow$  number of tokens,  $d \rightarrow$  token dimension

$$x_i \rightarrow i^{\text{th}} \text{ token} \in \mathbb{R}^d, W^{\{Q, K, V\}} \in \mathbb{R}^{d \times d}$$

$$q_i \rightarrow x_i W^Q, k_i \rightarrow x_i W^K, v_i \rightarrow x_i W^V$$

$$\text{probs}(p) \leftarrow \text{Softmax} \left( \text{Attention Scores} \leftarrow \frac{q_i K^T}{\sqrt{d}} \right)$$

$$\text{Attention}(a_i) \leftarrow pV$$

$$X \in \mathbb{R}^{n \times d}, Q, K, V \in \mathbb{R}^{n \times d}$$

$$Q \leftarrow XW^Q, K \leftarrow XW^K, V \leftarrow XW^V$$

$$\text{Attention} \leftarrow \text{Softmax} \left( \frac{QK^T}{\sqrt{d}} \right) V$$

# Transformer

- Three vectors (**query**( $q_i$ ), **key**( $k_i$ ), **value**( $v_i$ )) are generated from each input token( $x_i$ )
- Each token **queries** ( $q$ ) how much of **attention** it should give to every other token
- In **Self-Attention**, query and key, value belong to the same sequence
- In **Cross-Attention**, query belongs to a different sequence
- **Attention** layers are computationally less expensive than recurrent and convolutional layers

$X \rightarrow$  input,  $n \rightarrow$  number of tokens,  $d \rightarrow$  token dimension

$$x_i \rightarrow i^{\text{th}} \text{ token} \in \mathbb{R}^d, W^{\{Q, K, V\}} \in \mathbb{R}^{d \times d}$$

$$q_i \rightarrow x_i W^Q, k_i \rightarrow x_i W^K, v_i \rightarrow x_i W^V$$

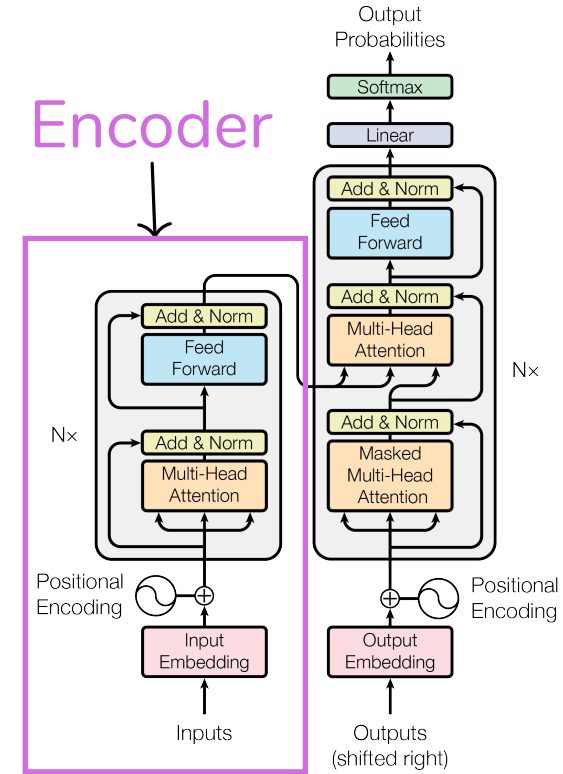
$$\text{probs}(p) \leftarrow \text{Softmax} \left( \text{Attention Scores} \leftarrow \frac{q_i K^T}{\sqrt{d}} \right)$$

$$\text{Attention}(a_i) \leftarrow pV$$

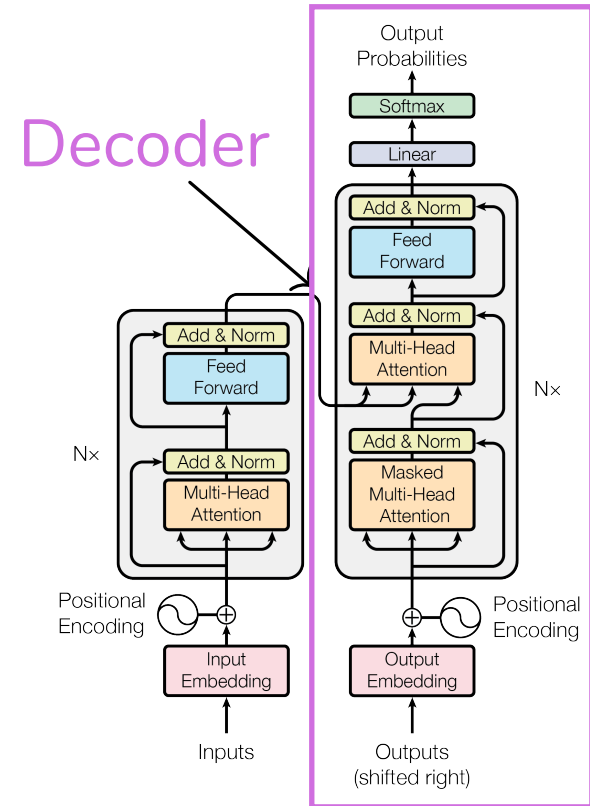
$$\begin{aligned} X &\in \mathbb{R}^{n \times d}, Q, K, V \in \mathbb{R}^{n \times d} \\ Q &\leftarrow XW^Q, K \leftarrow XW^K, V \leftarrow XW^V \\ \text{Attention} &\leftarrow \text{Softmax} \left( \frac{QK^T}{\sqrt{d}} \right) V \end{aligned}$$



# Transformer

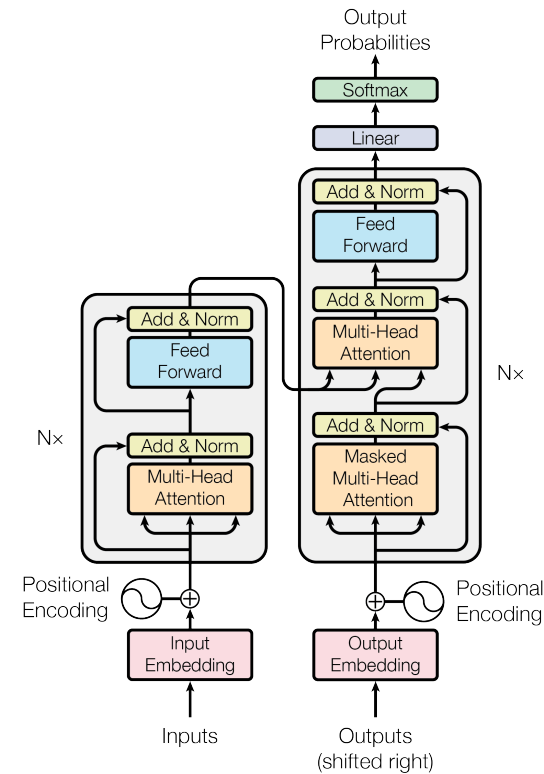


# Transformer



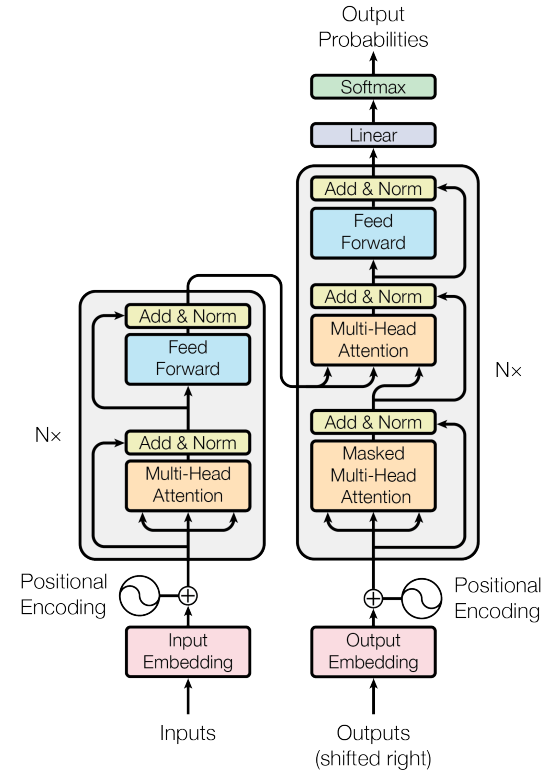
# Transformer

- In the **attention module** the attention computed is added to the input to incorporate contextual information



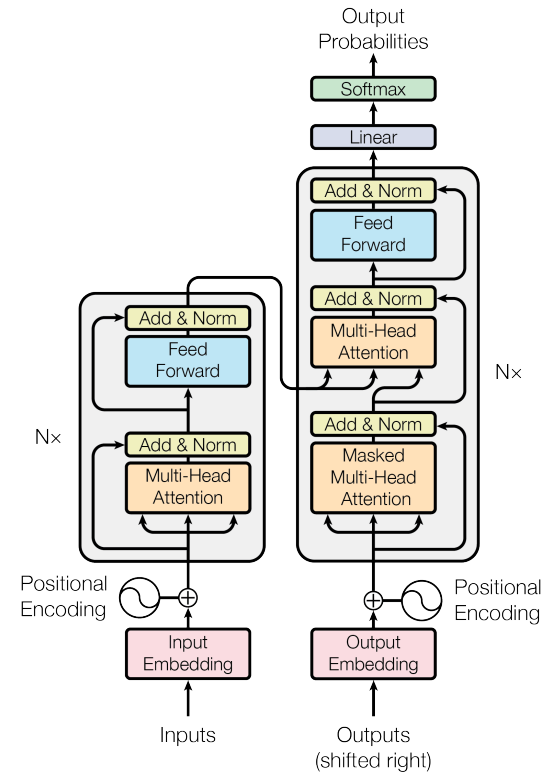
# Transformer

- In the **attention module** the attention computed is added to the input to incorporate contextual information
- **Normalization** and **Dropout** layers are added for regularization



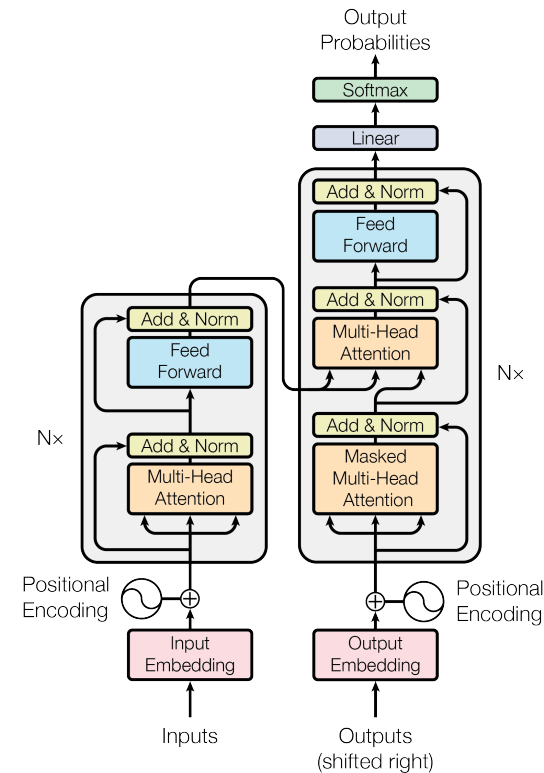
# Transformer

- In the **attention module** the attention computed is added to the input to incorporate contextual information
- **Normalization** and **Dropout** layers are added for regularization
- **Feed Forward Networks** add non-linearity to the model



# Transformer

- In the **attention module** the attention computed is added to the input to incorporate contextual information
- **Normalization** and **Dropout** layers are added for regularization
- **Feed Forward Networks** add non-linearity to the model
- Multiple **Attention + FFN** layers in sequence form the backbone of the transformer



# Transformer

- **Multi-Head** allows each head to attend to different feature
- **Masking** allows hiding information from the model
- It allows the model to process variable sized input
- It also works as a regularizer

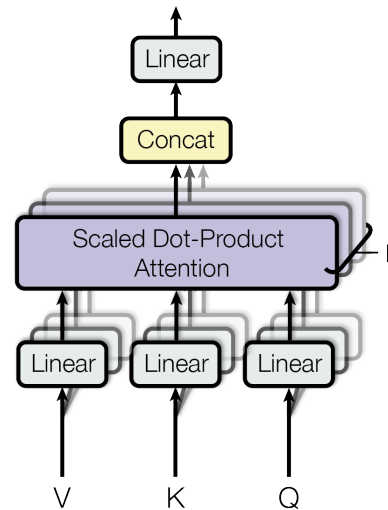


Figure 1: Multi-Head

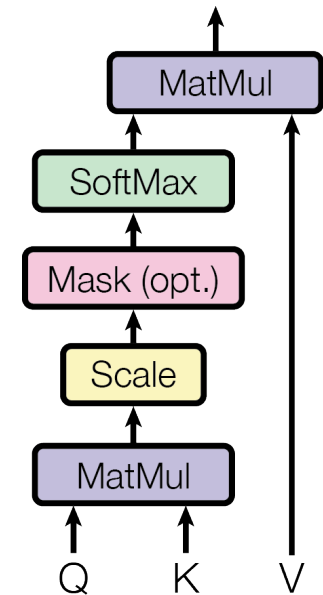


Figure 2: Scaled Dot-Product Attention

# Transformer

Absolute Positional Encoding [1] (add to the embedding)

$$\text{PE}_{\text{pos},i} = \begin{cases} \sin\left(\frac{\text{pos}}{1000^{\frac{i}{d_{\text{model}}}}}\right) & \text{if } i \text{ is even} \\ \cos\left(\frac{\text{pos}}{1000^{\frac{i}{d_{\text{model}}}}}\right) & \text{if } i \text{ is odd} \end{cases}$$
$$\text{pos} \leftarrow [0, \text{seq\_length}), i \leftarrow \left[0, \frac{d_{\text{model}}}{2}\right)$$

Rotary Positional Encoding [2] (rotate the embedding)

$$\theta_i = 10000^{-\frac{2i}{d_{\text{model}}}}, i \in \left[1, \frac{d_{\text{model}}}{2}\right]$$

- Effect of positional embedding decreases from the first dimension to the last dimension
- Early dimensions preserve the position information and higher dimensions preserve the token information
- This helps the model to choose which information to attend more



Contents

# QFormer

# Diffusion

# Bibliography

- [1] A. Vaswani *et al.*, “Attention is all you need,” in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, 2017, pp. 6000–6010.
- [2] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, “RoFormer: Enhanced Transformer with Rotary Position Embedding.” [Online]. Available: <https://arxiv.org/abs/2104.09864>